Kingdom of Saudi Arabia
Ministry of Education
Umm Al-Qura University
031

المملكة العربية السعودية
وزارة التعليم
جامعة أم القرى
٠٣١

AI Project
Project Title: Bank Marketing Dataset

Group members:

Name	ID
Rawan Qader	445013919
Asail hamdan	445000051
Atheer Alfahmi	443001237
Hajer Sayah	445013975

Task Distribution Table:

Name	Task Distribution
Rawan Qader	OneR & AdaBoostM1 & Bagging & analysis for each model I used & conclusion
Asail hamdan	BayesNet & Naive Bayes & IBK & Introduction: The purpose project, the learning objective and summary of the report the graph and its description
Atheer Alfahmi	Random forest, Stacking, SMO. The description about the dataset, personal analysis for the models.
Hajer Sayah	REPT TREE & JRip & J48 & Arranging comparison tables & data cleaning & reference

supervision by: Dr. Amirah Alharbi

1- Introduction:

1.1 Purpose of the Project:

The purpose of this project is to apply machine learning techniques to better understand the factors that influence whether a customer will subscribe to a term deposit. By analysing patterns in the data, we aim to help improve marketing strategies and target the right customers more effectively.

1.2 Learning Objectives:

This project provides an opportunity to strengthen our practical skills in working with real data. We'll focus on cleaning and preparing the dataset, experimenting with different machine learning models, and evaluating their performance in a meaningful way Additionally, it will help us build our ability to interpret model results, understand feature importance, and draw conclusions that support business decision-making. We'll also gain experience handling class imbalance, selecting suitable models, and visualizing results skills that are essential when working with real-world marketing data.

1.3 Summary of the Report:

This project uses a marketing dataset from a Portuguese bank to predict whether a customer will subscribe to a term deposit. We applied machine learning techniques, focused on classification models, and evaluated their performance. The goal is to support better marketing decisions using data-driven insights.

2- The Goal

The ultimate goal is to build a predictive model that can accurately classify customers into two groups: those who are likely to subscribe and those who are not. This can support banks in making smarter marketing decisions and improving their campaign results.

3- About the Dataset

The dataset selected for this project is the **Bank Marketing** Dataset obtained from the **UCI Machine Learning Repository**.

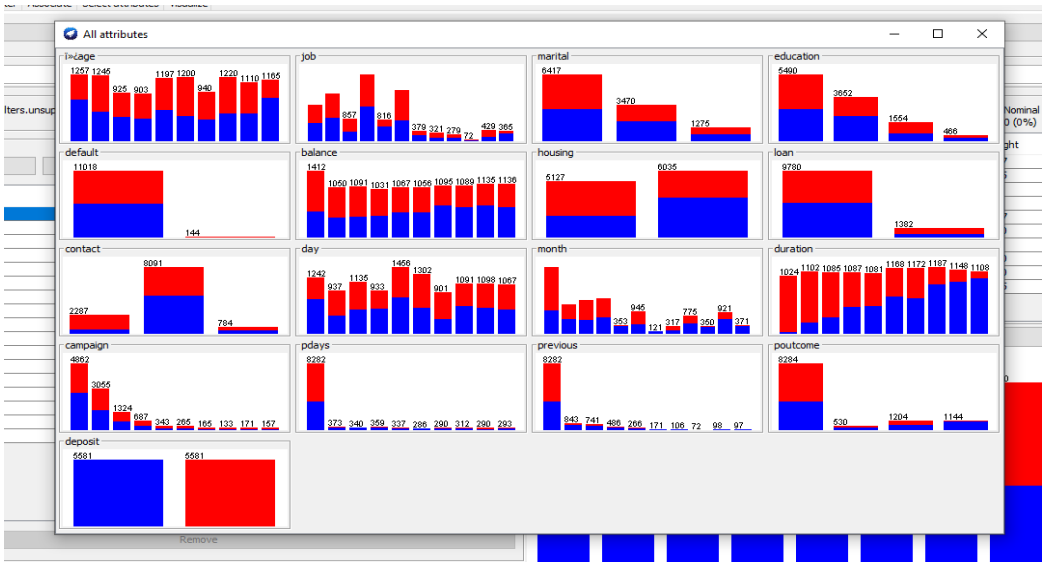
This dataset contains comprehensive customer-related data collected from various banking campaigns, encompassing:

- **Demographic Details**: Age, job, marital status, and education level of the clients.
- **Socioeconomic Factors**: Information such as housing loan status, personal loan, and account balance, which may influence financial behavior.
- **Contact Information**: Details about the type and outcome of marketing contacts.
- **Campaign Details**: Records of marketing efforts (e.g., number of contacts, days since last contact, and duration of contact) provided to the clients.
- **Outcomes**: Information on whether the client subscribed to a term deposit.

With data from **thousands of clients**, this dataset is structured to support detailed statistical and machine-learning analyses. It serves as a valuable resource for researchers aiming to identify customer segments, predict campaign outcomes, and generate actionable insights for improving bank marketing strategies. The dataset contains a total of **11162 instances** and **17 attributes**, such as age, job, and other personal or contact details of the clients.

Feature	Description
Age	Age of the client (numeric).
Job	Type of job (e.g., admin., technician, entrepreneur, etc.).
Marital	Marital status of the client (e.g., married, single, divorced).
Education	Education level of the client (e.g., primary, secondary, tertiary, unknown).
Default	Whether the client has credit in default (yes/no).
Balance	Average yearly balance in euros (numeric).
Housing	Whether the client has a housing loan (yes/no).
Loan	Whether the client has a personal loan (yes/no).
Contact	Type of communication contact (e.g., cellular, telephone).
Day	Last contact day of the month (numeric).
Month	Last contact month of the year (e.g., jan, feb, mar, etc.).
Duration	Duration of the last contact in seconds (numeric).
Campaign	Number of contacts performed during this campaign (numeric).
Pdays	Number of days since the client was last contacted (-1 means not previously).
Previous	Number of contacts performed before this campaign (numeric).
Poutcome	Outcome of the previous marketing campaign (e.g., success, failure, unknown).
Deposit	Whether the client subscribed to a term deposit (yes/no).

Data Cleaning



Missing values, outliers, and class imbalance were among the first problems in the data that impacted on the model's quality to guarantee the best data quality we thus went through a cleaning phase with WEKA filters

Unsupervised → Attribute → ReplaceMissingValues

It is a filter to fill in the blanks in columns like default, education, and employment.
The filter substitutes a suitable statistical value (like the median or the most frequent value) for missing values.

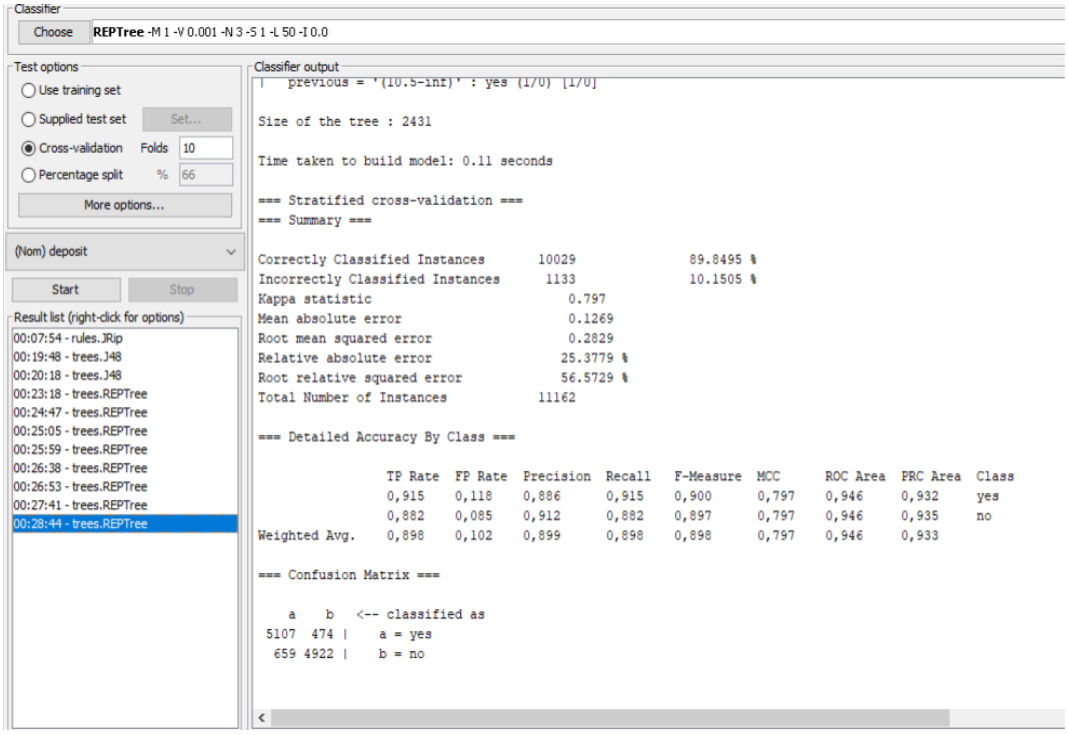
Unsupervised → Attribute → Discretize

It was applied to numeric columns such as age, balance, duration, pdays, and campaign to classify them into categories. It helped improve the accuracy of classification algorithms such as J48 and Naive Bayes.

Supervised → Instance → Resample

The target class deposit was unbalanced (mostly "no"), which impacted on the model's performance.
Resample was used to generate a balanced distribution between the two classes, which helped improve recall for the "yes" class and made the model more predictive.

• REPT TREE



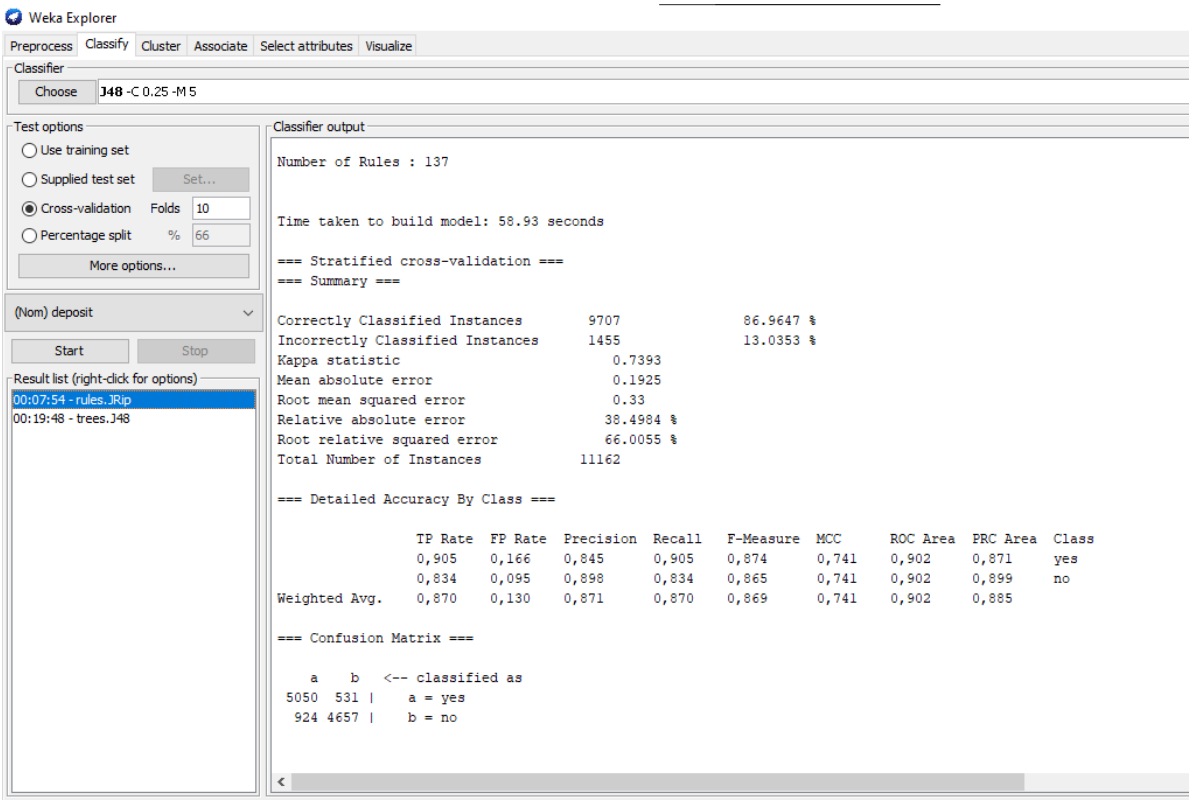
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	89.8495%	0 .886	0.915	0.900	0.946
No		0.912	0.882	0.897	0.946
Avg		0.899	0.898	0.898	0.946

Confusion Matrix		
Classified as	a	b
a = yes	5107	474
b = no	659	4922

Personal Analysis:

In order to give the model a detailed depth, I reduced **MinNo** to 1. I allowed the tree to use deep structures to find every detail in the data by setting a large **MaxDepth** (50). Because the number of **folds** 3 allowed for internal validation of the model, this modification improved the classification ability
without overfitting & it was the highest accuracy.

• JRip



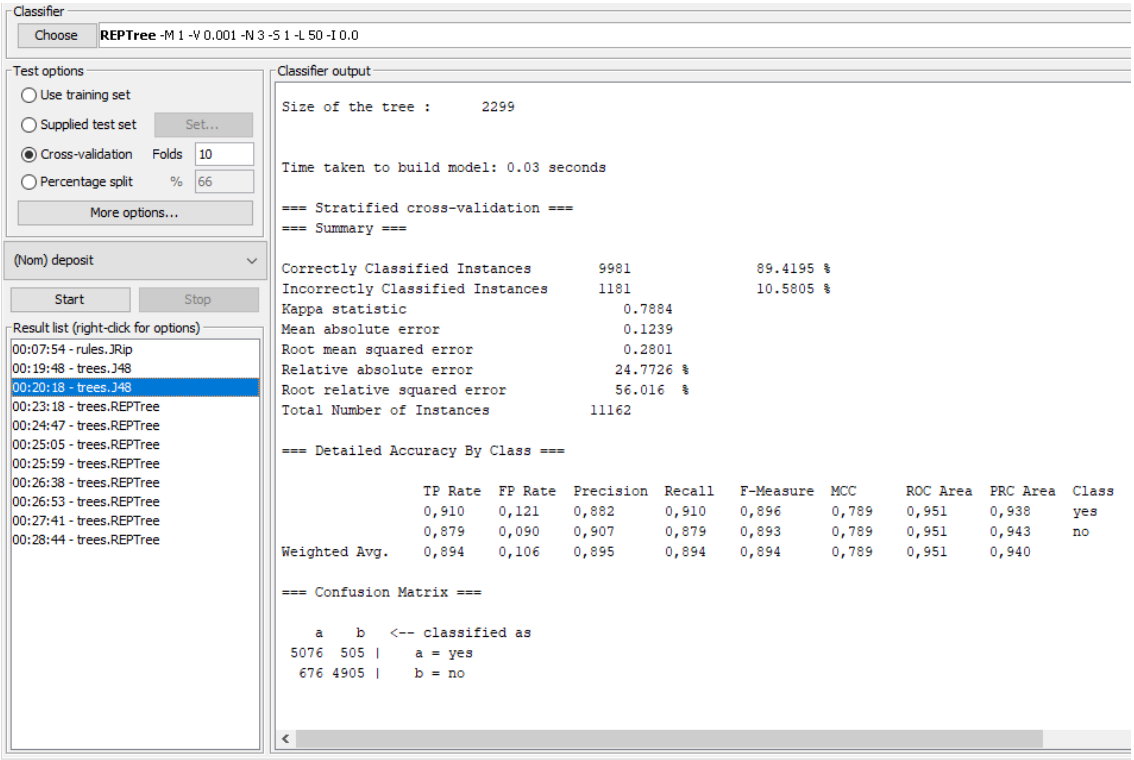
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	86.9647%	0 .845	0.905	0.874	0.902
no		0.898	0.834	0.865	0.902
Avg		0.871	0.870	0.869	0.902

Confusion Matrix		
Classified as	A	b
a = yes	5050	531
b = no	924	4657

Personal Analysis:

I used a small **MinNo** value (2) to allow the model to generate more precise and detailed rules. This is very useful when I have data with a specific pattern for a small class, such as "yes." A few **optimizations** (2) helped reduce training time while maintaining performance. Choosing 3 **Folds** is a good balance between speed and accuracy.

- **J48**



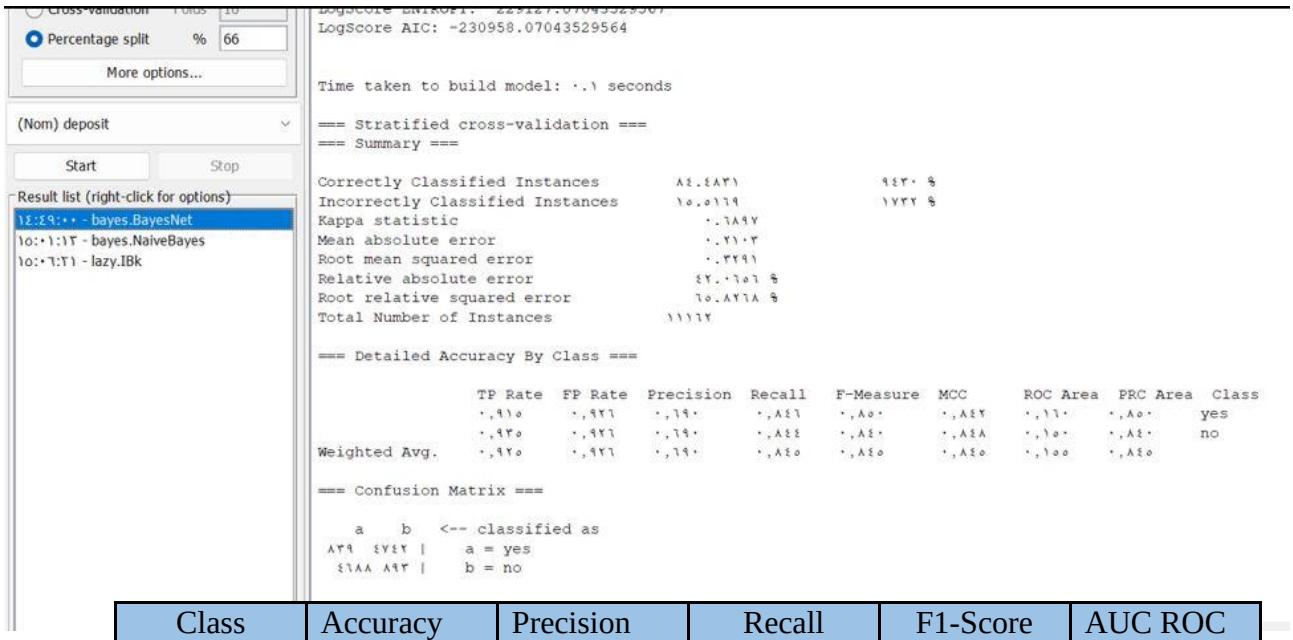
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	89.4195%	0 .882	0.910	0.896	0.951
no		0.907	0.879	0.893	0.951
Avg		0.895	0.894	0.894	0.951

Confusion Matrix		
Classified as	a	b
a = yes	5076	505
b = no	676	4905

Personal Analysis:

The model had a lot of freedom to explore every branch since the tree could be constructed with no pruning (**Unpruned** = True). Noise from tiny data points was decreased by increasing **MinNumObj** 5. A **factor** of 0.25 provided an appropriate balance between complexity and ease of use.

• BayesNet



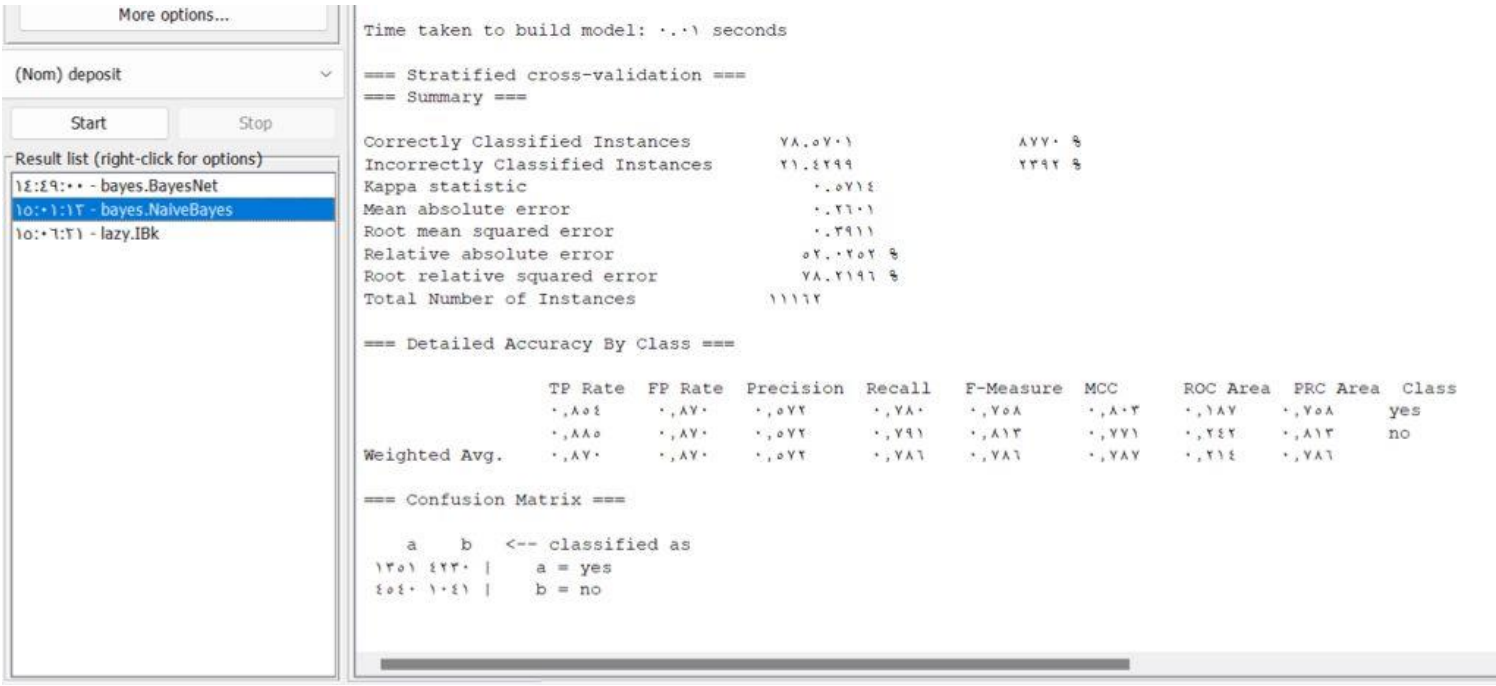
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	84.4831%	0 .690	0.846	0.850	0.160
no		0.690	0.844	0.840	0.150
Avg		0.690	0.845	0.848	0.155

Confusion Matrix		
Classified as	a	b
a = yes	839	4742
b = no	893	4688

Personal Analysis:

I modified the **estimator** from **0.5 to 2.0** to give the model more flexibility in handling the data, and I also changed the **search algorithm from K2 to TAN**, which improved the network structure and boosted accuracy. I kept all the other settings unchanged, and these simple adjustments led to better performance for our complex dataset.

• NaiveBayes



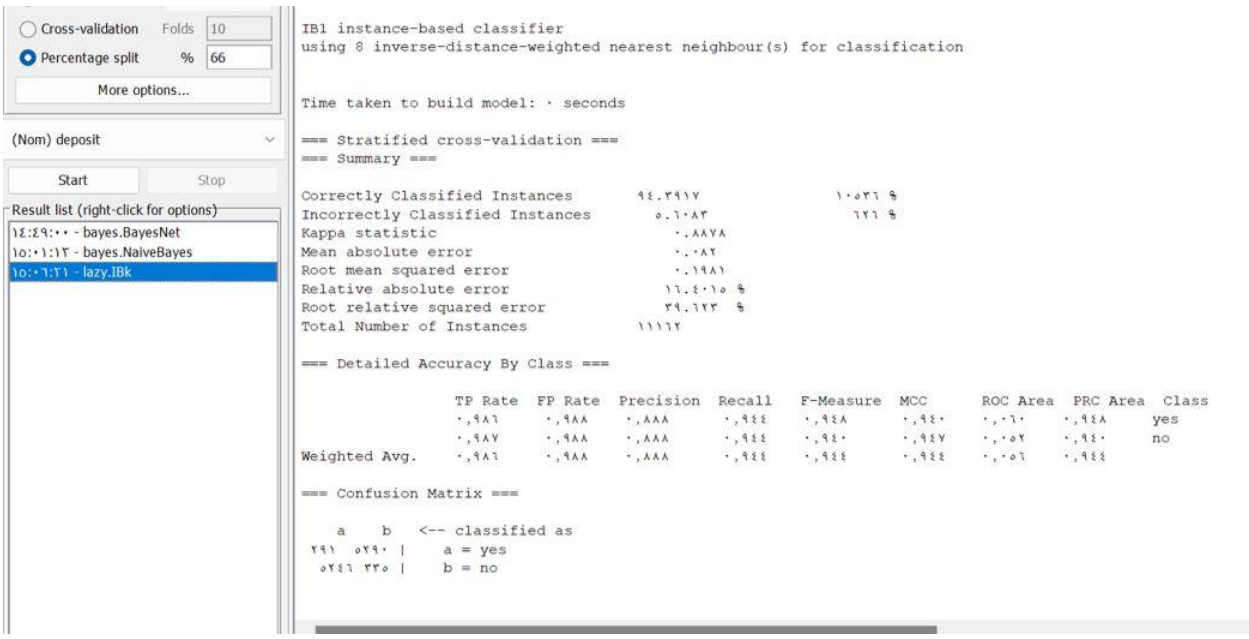
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	78 .5701%	0 .572	0.780	0.758	0.187
no		0.572	0.791	0.813	0.242
Avg		0.572	0.786	0.786	0.214

Confusion Matrix		
Classified as	a	b
a = yes	1351	4230
b = no	4540	1041

Personal Analysis:

I experimented with several parameter settings in the Naive Bayes classifier and found that disabling the **kernel estimator (setting it from true to false)** significantly improved performance. By keeping all other default settings unchanged, this simple adjustment led to a more accurate and stable model for our dataset.

• IBK



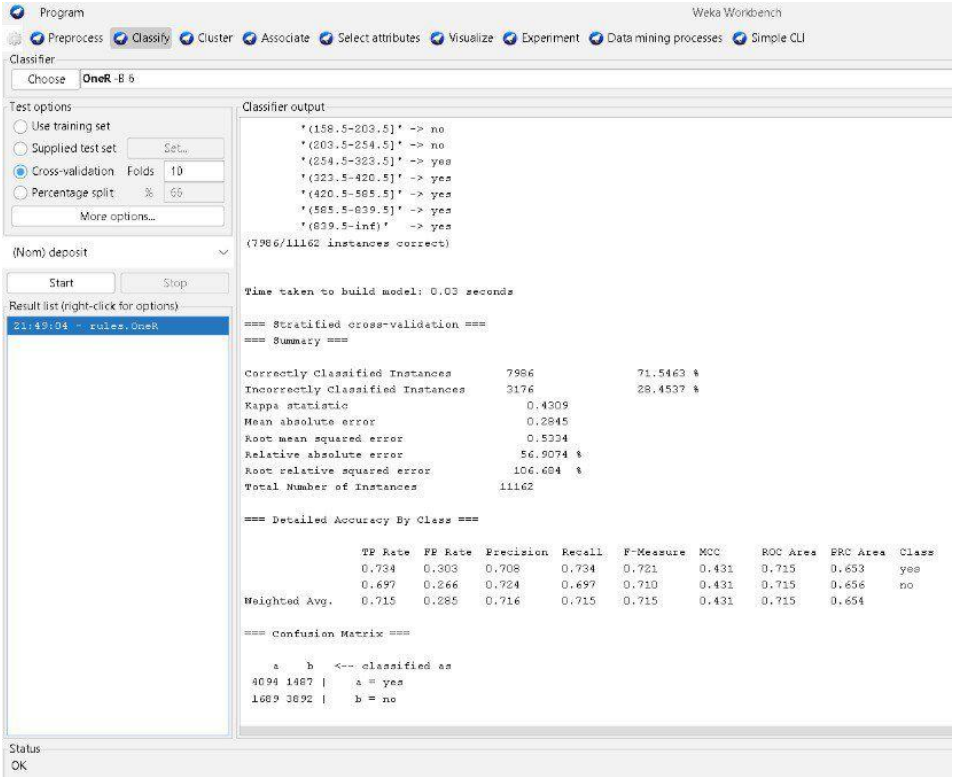
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	94.3917%	0 .888	0.944	0.948	0.060
NO		0.888	0.944	0.940	0.052
Avg		0.888	0.944	0.944	0.056

Confusion Matrix		
Classified as	a	b
a = yes	291	5290
b = no	5246	335

Personal Analysis:

I tested multiple K values (1, 2, 3, 4, and 8) and found that K = 8 gave the best performance for our dataset. I also changed the distance weighting from “no distance weighting” to “weight by 1/distance”, which helped the model give more importance to closer neighbors All other settings were kept as default. This configuration not only improved accuracy but also outperformed the previous two algorithms (Naive Bayes and BayesNet) in terms of overall performance

• OneR



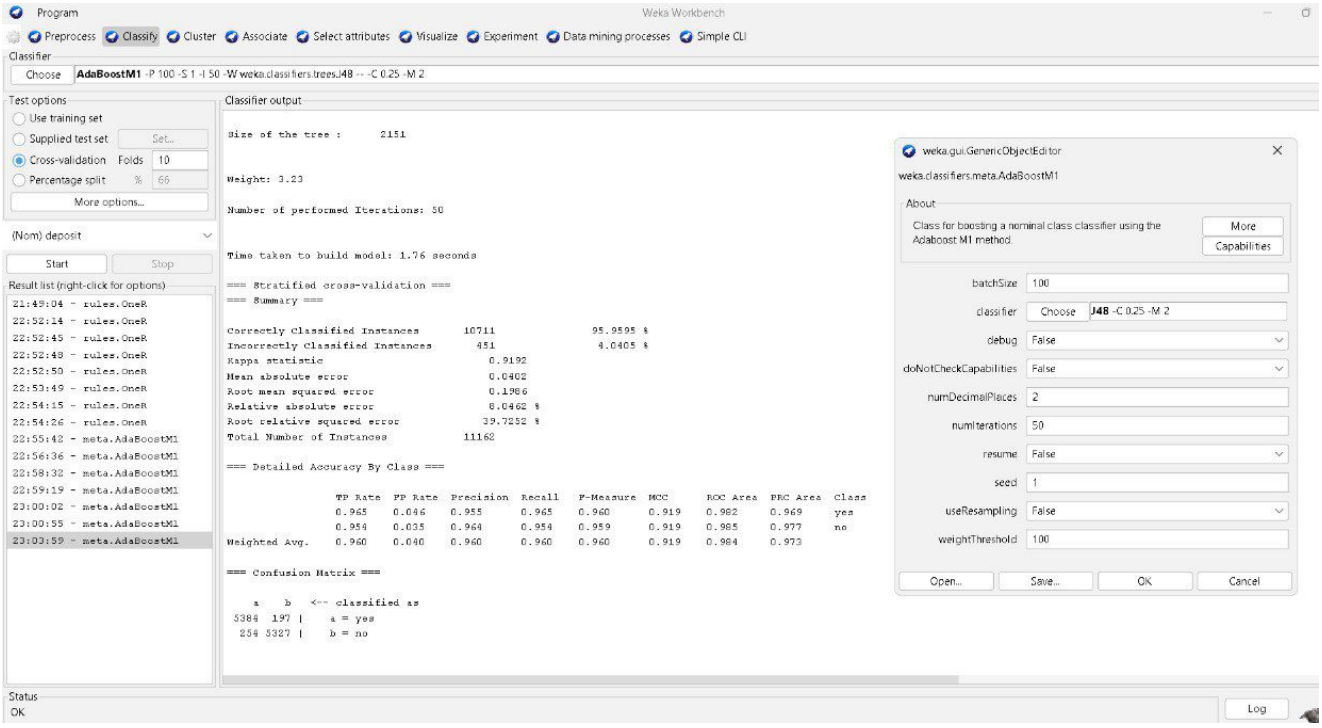
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	71.5463%	0 .708	0.734	0.721	0.715
no		0.724	0.697	0.710	0.715
Avg		0.716	0.715	0.715	0.715

Confusion Matrix		
Classified as	a	b
a = yes	4094	1487
b = no	1689	3892

Personal Analysis:

OneR gave an accuracy of 71.5463% with the default minBucketSize = 6. Changing it to 2 or 10, and adjusting batchSize had no effect. OneR is simple and fast but limited, as it builds rules using only one attribute.

• AdaBoostM1



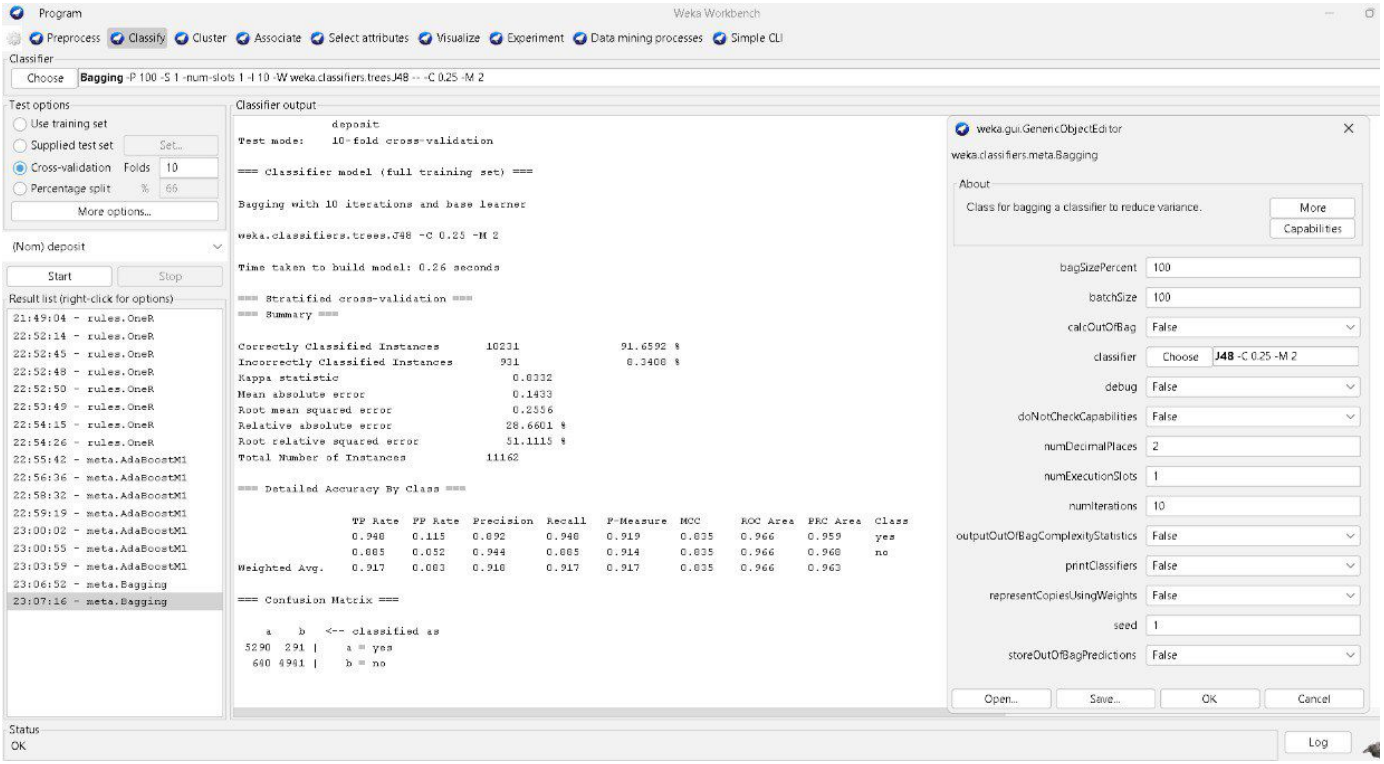
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	95.9595%	0 .955	0.965	0.960	0.982
no		0.964	0.954	0.959	0.977
Avg		0.960	0.960	0.960	0.973

Confusion Matrix		
Classified as	a	b
a = yes	5384	197
b = no	254	5327

Personal Analysis:

The default accuracy was 71.6269% with numIterations = 10. Increasing to 50 improved accuracy to 81.3922% and to 83.0138% with 100. The biggest jump came when switching the base classifier to J48, reaching 95.9595%. NumIterations controls how many boosting rounds are done; using J48 made the model much stronger.

• Bagging



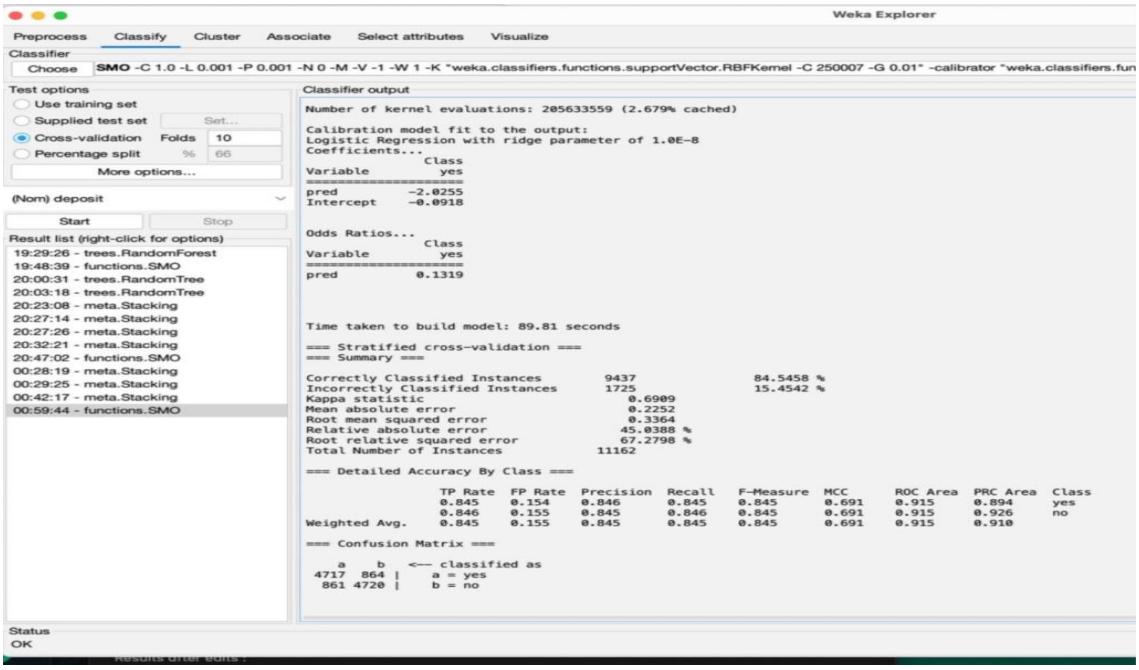
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	91.6592%	0 .892	0.940	0.919	0.966
no		0.944	0.005	0.914	0.966
Avg		0.910	0.917	0.917	0.966

Confusion Matrix		
Classified as	a	b
a = yes	5290	291
b = no	640	4941

Personal Analysis:

Bagging achieved 91.6592% accuracy using J48 as the base classifier, with 100 Iterations and a bag size of 100%. It builds multiple models on different subsets of data and averages their results, improving accuracy and reducing overfitting. It was one of the most effective models in the project.

- **SMO**



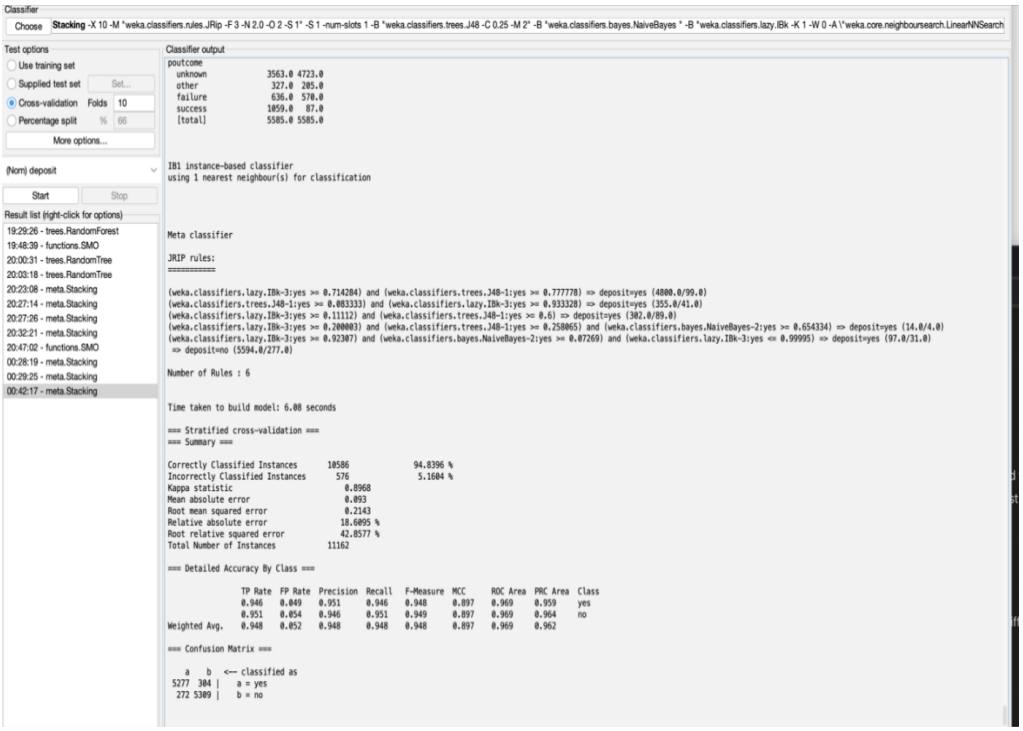
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	84.5458%	0 .846	0.845	0.845	0.915
no		0.845	0.846	0.845	0.915
Avg		0.845	0.845	0.845	0.915

Confusion Matrix		
Classified as	a	b
a = yes	4717	864
b = no	861	4720

Personal Analysis:

I used the RBF kernel to help the model find more complex patterns in the data. I set C to 1.0 for a good balance between accuracy and simplicity. I turned on calibration to get better probability results. I also set epsilon and tolerance to 0.001to make training faster. These changes helped the model perform better overall

• Stacking



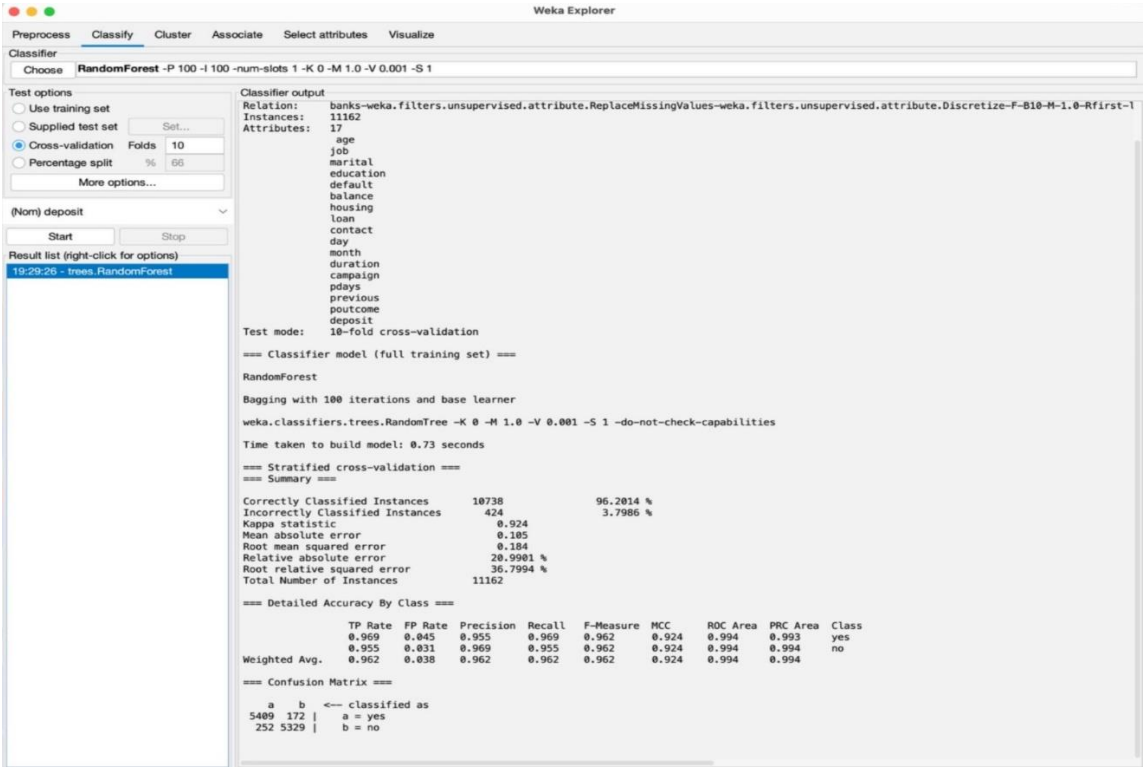
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	94.8396%	0 .951	0.946	0.948	0.969
no		0.946	0.951	0.949	0.969
Avg		0.948	0.948	0.948	0.969

Confusion Matrix		
Classified as	a	b
a = yes	5277	304
b = no	272	5309

Personal Analysis:

I used a stacking method that combines J48, Naive Bayes, and IBK classifiers to improve overall accuracy. On top of them, I added JRip as a meta-classifier to generate easy-to-follow rules. The model reached 94.8% accuracy using 10-fold cross-validation and performed well for both “yes” and “no” outcomes. Even with multiple classifiers, it was fast and efficient, building the model in just a few seconds.

- **Random forest**



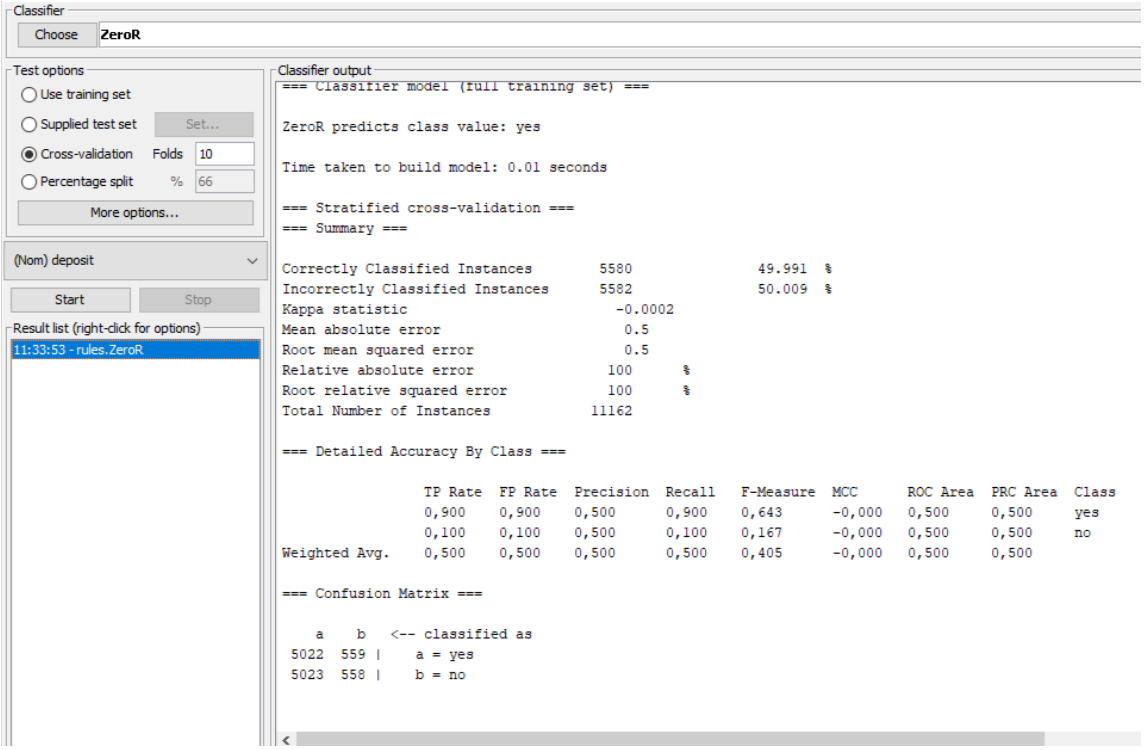
Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	96.2014%	0 .955	0.969	0.962	0.994
no		0.969	0.955	0.962	0.994
Avg		0.962	0.962	0.962	0.994

Confusion Matrix		
Classified as	a	b
a = yes	5409	172
b = no	252	5329

Personal Analysis:

I used the RandomForest model with 100 trees to help find detailed patterns in the data, especially for the "yes" class. I tested it using 10-fold cross-validation and got a high accuracy of 96.2%. The model was also very good at telling the difference between the classes. Even with these detailed settings, it trained quickly.

• ZeroR



Class	Accuracy	Precision	Recall	F1-Score	AUC ROC
Yes	49.991%	0 .500	0.900	0.643	0.500
no		0.500	0.100	0.167	0.500
Avg		0.500	0.500	0.405	0.500

Confusion Matrix		
Classified as	a	b
a = yes	5022	559
b = no	5023	558

Hajer algorithms:

Name	Accuracy
REPT TREE	89.8495 %
JRip	86.9647%
J48	89.4195%
The Best REPT	**One is TREE**

Asail algorithms:

Name	Accuracy
BayesNet	84.4831 %
NaiveBayes	78.5701%
IBK	94.3917%
The Best IBK	**One is**

Rawan algorithms:

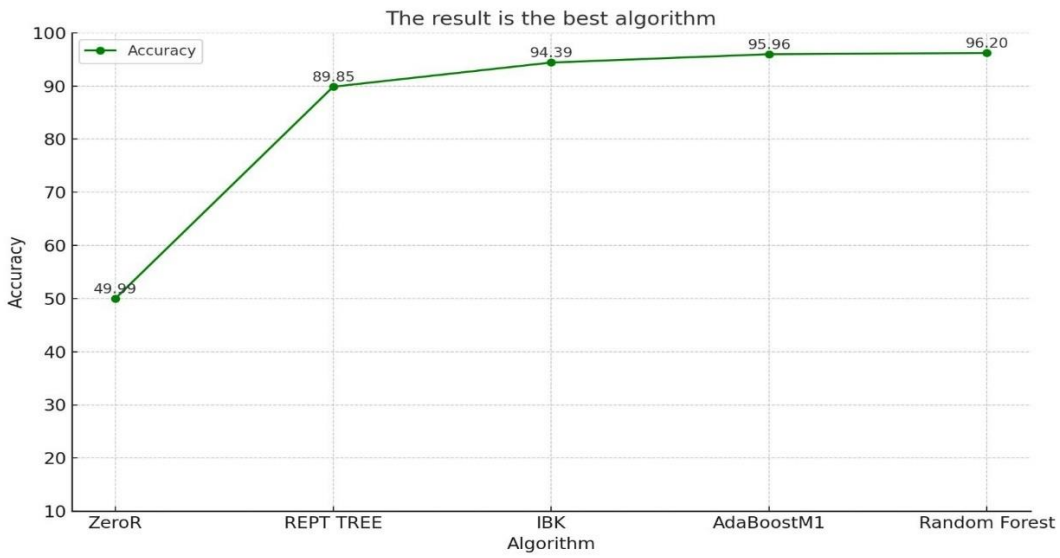
Name	Accuracy
OneR	71.5463%
AdaBoostM1	95.9595%
Bagging	91.6592%
The Best AdaBoostM1	**One is**

Atheer algorithms:

Name	Accuracy
SMO	84.5458 %
Stacking	94.8396%
Random forest	96.2014%
The Best Random Forest	**One is**

All tests were performed using cross-validation.

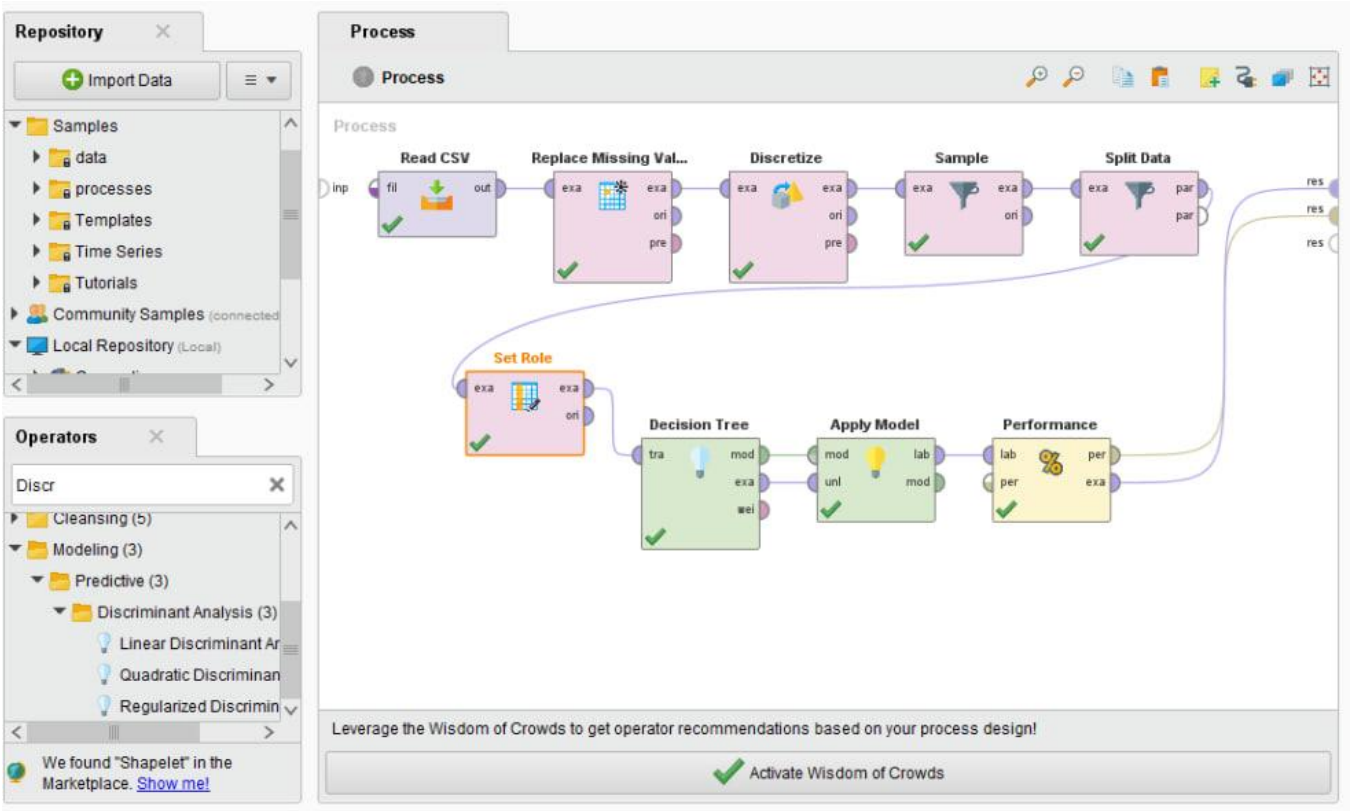
Name	Accuracy
REPT TREE	89.8495 %
IBK	94.3917%
Random forest	96.2014%
AdaBoostM1	95.9595%
ZeroR	49.991%
The Best One is Random Forest	

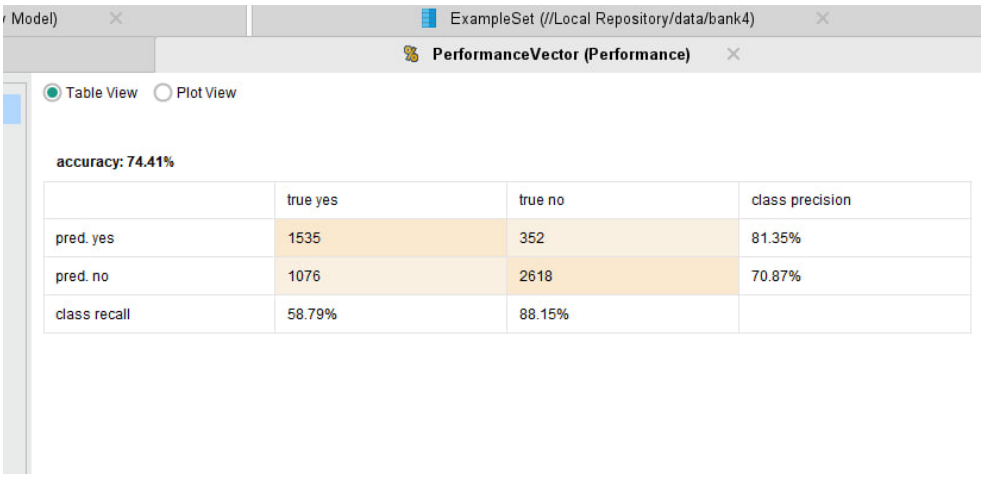


The Random Forest algorithm outperformed other models in this project because it combines the results of multiple decision trees built randomly with resampling, which reduces bias and improves the model's generalization ability. Thanks to the diversity of features in the bank dataset—demographic, economic, and contact-related—Random Forest was able to capture complex patterns and achieve the highest accuracy of 96.2%.

We tried using RapidMiner app and the results were:

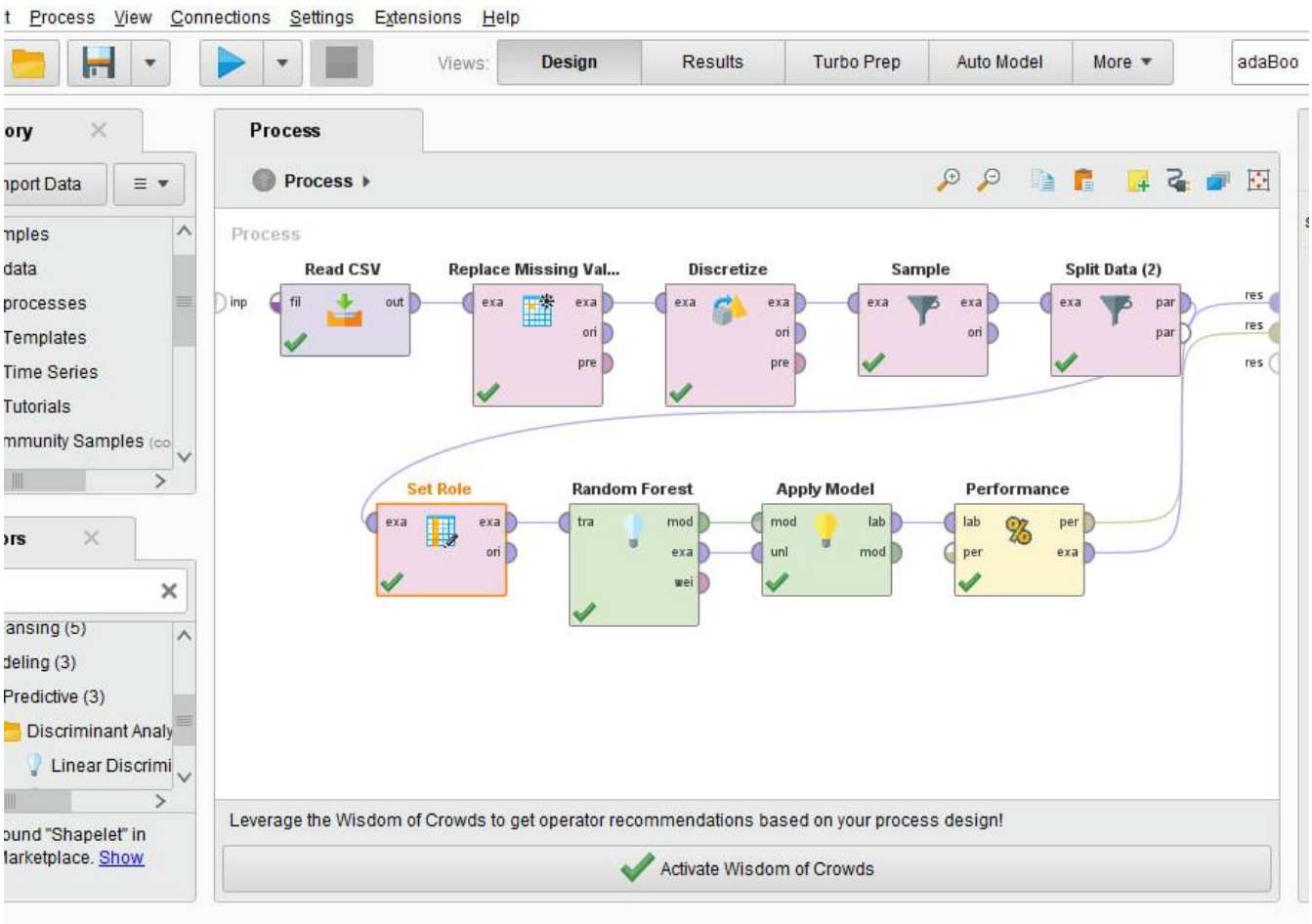
- **Decision Tree**

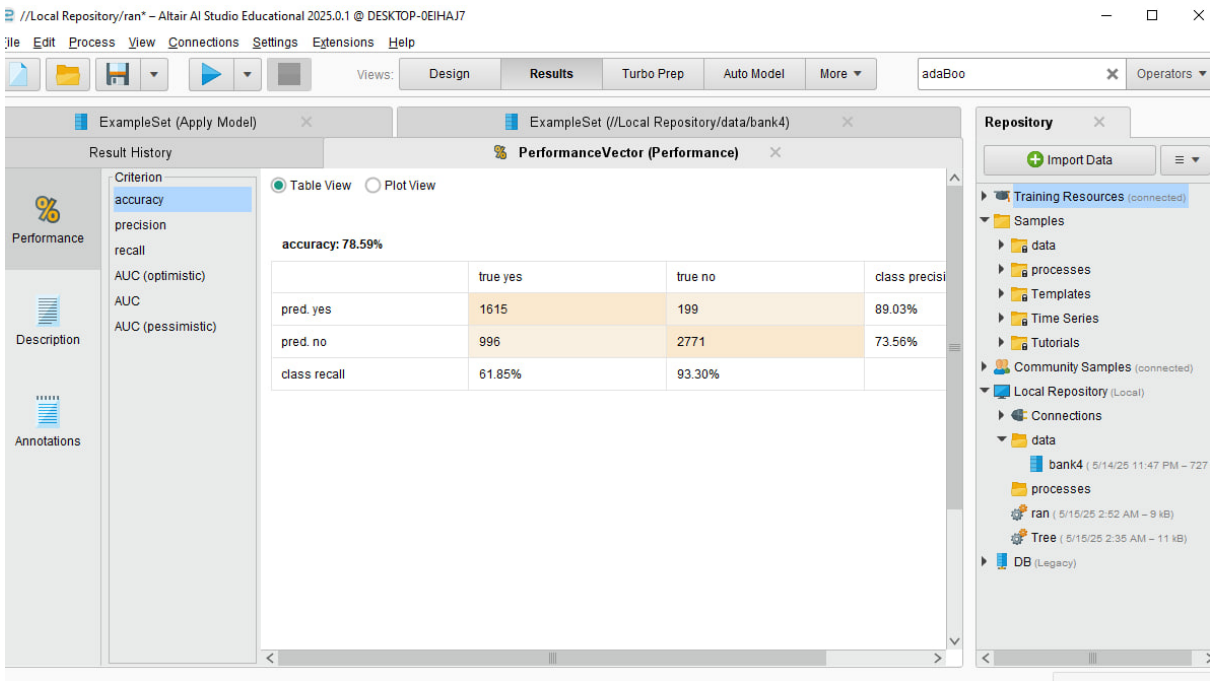




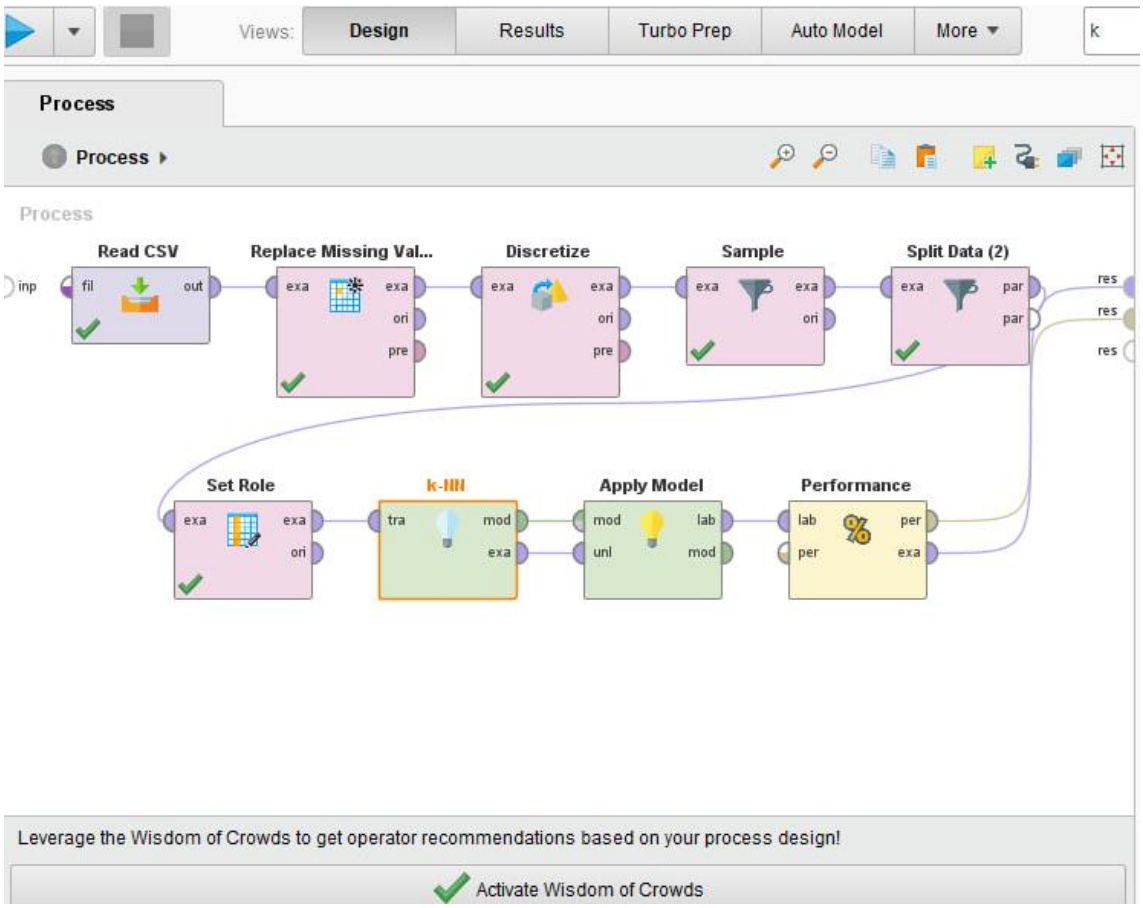
	true yes	true no	class precision
pred. yes	1535	352	81.35%
pred. no	1076	2618	70.87%
class recall	58.79%	88.15%	

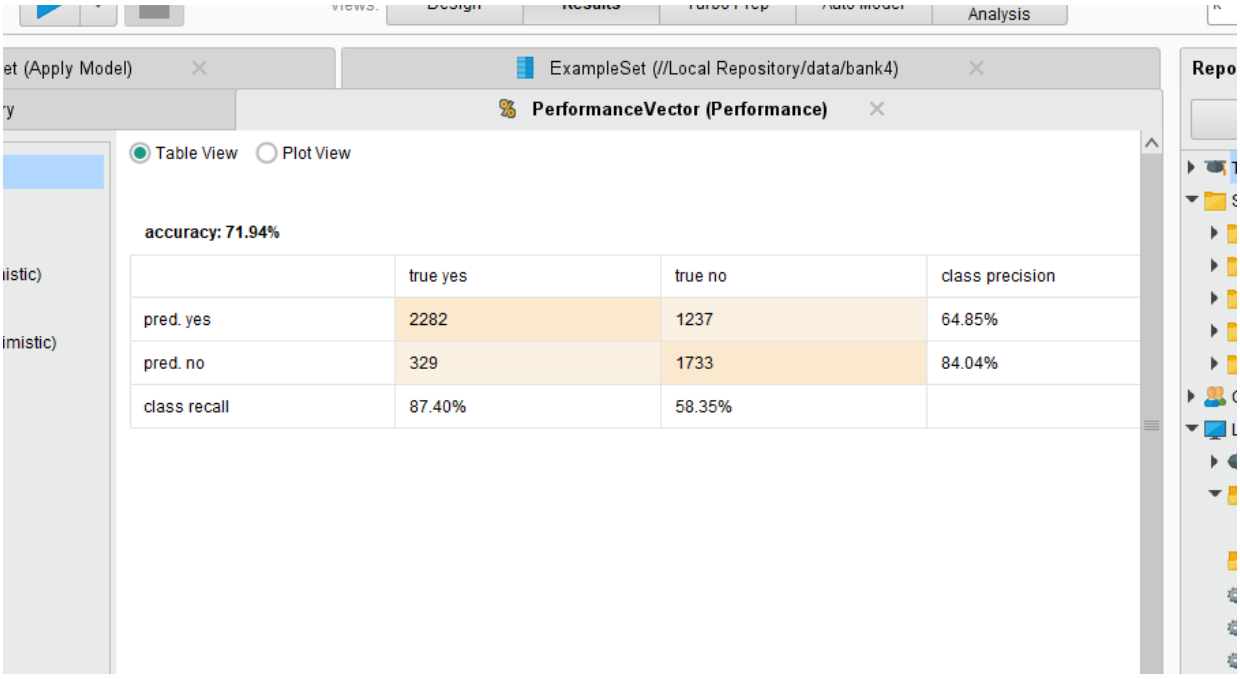
- **Random Forest**





- **K-NN**





	true yes	true no	class precision
pred. yes	2282	1237	64.85%
pred. no	329	1733	84.04%
class recall	87.40%	58.35%	

Conclusion:

Through this project, we were able to explore and analyze real-world data from banking marketing campaigns using various machine learning techniques. The models applied helped us evaluate the key factors influencing whether customers subscribe to term deposits. We enhanced model performance by handling missing values, discretizing numerical data, and balancing class distribution.
The results showed that the **Random Forest** algorithm achieved the highest accuracy of 96.2014%, making it a powerful tool for supporting data-driven marketing decisions. This project not only improved our technical skills but also deepened our understanding of applying machine learning in real business contexts

reference:

https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset?resource=download&select=bank.csv

https://machinelearningmastery.com/how-to-handle-missing-values-in-machine-learning-data-with-weka/

https://www.tutorialspoint.com/weka/weka_classifiers.htm

https://www.geeksforgeeks.org/introduction-to-weka-key-features-and-applications/

https://www.youtube.com/playlist?list=PL9FfW__Sm3pIyyakq22P2mMi82neT2LWS&si=RaWBtUHxXxRo8Vw-

https://www.youtube.com/playlist?list=PL86JZGzAEh4ud3JaPNvU3EFD_Z22peVSH&si=cgEujXx5-u8POJMh